

Homework 3: Dynamic Programming 2D

Guideline

Homework chatbot. [Open the homework Gem here](#). Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) [Unregulated usage of AI harms learning](#) (2) Even in the age of AI, your [AI-independent ability matters](#). The **only** AI tool you should use on this homework is the homework-specific [Gemini Gem](#) that I have created for you.

The 4-step cookbook. Same recipe as always:

1. **Define subproblems.** `dp[i][j]` = "best" way to handle some prefix of each input
2. **Find recurrence.** `dp[i][j] = f(dp[i-1][j], dp[i][j-1], dp[i-1][j-1], ...)`
3. **Specify order.** Which way do the loops run?
4. **Base case.** What are row 0 and column 0?

Submission. Upload all files to the Homework 3 assignment on Canvas. **5 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw3.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Four AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
 - `wordbreak-d.pdf`
 - `minpath-d.pdf`
 - `asciidelete-b.pdf`, `asciidelete-e.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, October 11, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 28 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare

yourself for the quizzes.

1. Word break redux (6 pts total)

In Homework 2 you wrote a word break solution that answered **true or false**. Now make it tell you *which words*.

Extend your Homework 2 code so that it prints the actual segmentation to the console. If you never got Homework 2 problem 4 working, that's fine -- start from the posted Homework 2 solution instead.

[Leetcode link](#)

Your program must handle these three cases:

#	s	wordDict
1	"starshipstar"	["star", "stars", "hip"]
2	"starshipper"	["star", "starship", "ship", "shipper"]
3	"starshi"	["star", "stars", "hip"]

Print the segmentation as an array of strings. If the string cannot be segmented at all, print `no segmentation`.

For case 1, your console must show exactly:

```
['stars', 'hip', 'star']
```

Each of these three cases has a **unique** segmentation (or none), so everyone's output should match.

(a) Planning

Do **not** use AI for this part.

(i) The second table (1 pt)

Your `dp` array only remembers *whether* a prefix is segmentable. To print the words you need to remember *how*. Describe the second data structure you will keep alongside `dp`: what is it indexed by, and what does a single entry hold?

(ii) Walking it backwards (1 pt)

Describe, in 2 to 4 sentences, how you recover the list of words from that structure once the table is filled. **Careful:** walking backwards gives you the words in reverse. Say how you handle that.

(b) Console output (1 pt)

Run your program on all three cases above and paste the **exact console output** for each into your answer document.

(c) Code (1 pt)

Turn in your final code, pasted into your answer document.

(d) AI code check

Copy paste your code to the [homework gem](#) and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Does my reconstruction print the words in the right order? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `wordbreak-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

2. Min path sum recap (10 pts total)

Let's code up the problem discussed in the lecture!

Given a grid of non-negative integers, find a path from the **top left** to the **bottom right** that minimizes the sum of the cells along it. You may only move **right** or **down**.

[Leetcode link](#)

Use this grid, which is **not** the one from lecture:

	c=0	c=1	c=2	c=3
r=0	1	3	1	2
r=1	2	1	4	1
r=2	5	2	1	3
r=3	1	2	1	1

(a) From memory

Do **not** use AI for this part. Refer to the lecture materials first, then close them and write from memory.

(i) Define the subproblem (1 pt)

Write down, in **one line**, what `dp[r][c]` means.

(ii) Find the recurrence (1 pt)

Express `dp[r][c]` in terms of smaller subproblems. Show the exhaustive cases you are taking the min over.

(iii) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences. Imagine the **optimal** path. Argue that it must arrive at each cell through one of your cases, and that within each case it must have reached that neighbor optimally.

(iv) Compute the `dp` table by hand (1 pt)

Compute a 2x2 dp table for just the top left corner. That is, compute dp values for (0,0), (1,0), (0,1), (1,1) by hand

(v) Runtime (1 pt)

What is the big-O runtime for an R by C grid, and why?

(vi) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

(b) Code, with reconstruction (1 pt)

Code it up, and extend it so that it also **prints the optimal path** to the console. Print the path as a comma-separated list of `(row, col)` tuples, starting at the top left.

For the **lecture** grid `[[1,3,1],[1,5,1],[4,2,1]]`, the output would be:

```
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]
```

Paste the exact console output your program produces for the 4x4 grid above. That grid has a **unique** minimum path, so everyone's output should match.

(c) Turn in code (1 pt)

Turn in your final code for generating the previous output, pasted into your answer document.

(d) AI code check

Copy paste your code to the [homework gem](#) and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Is my path reconstruction correct? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `minpath-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

3. Minimum ASCII delete sum (12 pts total)

Given two strings `s1` and `s2`, delete characters from either string until the two are equal. Each deletion costs the **ASCII value** of the character you delete. Return the lowest total cost. Note: in python, you can use the `ord` function.

[Leetcode link](#)

Our running example:

```
s1 = "sea", s2 = "eat"
```

The answer is `231`. Delete the `s` from `"sea"` (ASCII 115) and the `t` from `"eat"` (ASCII 116), leaving `"ea"` on both sides. $115 + 116 = 231$.

(a) No-AI-attempt planning

Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line**, what `dp[i][j]` means.

(ii) Find the recurrence (1 pt)

Express `dp[i][j]` in terms of smaller subproblems. Split into two cases, exactly as we did for LCS in lecture: the last characters **match**, and they **don't**. Show both.

(iii) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences. Focus on the matching case: argue why it is **never worse** to keep a pair of matching characters than to delete one or both of them.

(b) AI plan check

Use the [homework chatbot](#) to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my dp definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why is it never worse to keep a pair of matching characters?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `asciidelete-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Compute a **dp** cell by hand (1 pt)

For `s1 = "sea"`, `s2 = "eat"`, what is the min cost of making the **first two** characters of `s1` equal to the **first one** character of `s2` -- that is, `"se"` against `"e"`? Which dp cell in your definition corresponds to this -- (1,0), or (2,1)? ASCII values you will need: `e = 101`, `s = 115`

(d) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

Note: in python, you can use the `ord` function.

(e) AI code check

Copy paste your code to the [homework gem](#) and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `asciidelete-e.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(f) Runtime (1 pt)

What is the big-O runtime of your solution, and why?

(g) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

(h) Relate it back to LCS (1 pt)

Now instead of having variable deletion costs, suppose that every deletion costs exactly **1**, no matter which character it is (also [Leetcode 583](#)). Let's consider how this relates to the longest common subsequence problem discussed in the lecture.

Let:

- m = the length of s_1
- n = the length of s_2
- L = the length of the **longest common subsequence** of s_1 and s_2

Write down the minimum number of deletions, d , in terms of m , n and L , and explain in 2 to 3 sentences **why** solving LCS allows you to solve the min-deletion problem.